Datawhistl

Why Enterprise AI Agents Fail Beyond The Demo

And how enforceable architecture standards come to your rescue.

DHEERAJ SAXENA

MD@ DATAWHISTL.COM

ABOUT

1 25+ yrs. experience in enterprise solutions consulting, last 5 yrs in Enterprise AI Architecture and Advisory

AI Enterprise Architecture, AI & Data Readiness Assessments.

2 Enterprise data consulting heritage

Technology Strategy & Architecture Leadership for multi-million-dollar enterprise data platforms — covering feature stores, vector databases, and customer data platforms.

CONTACT ME

+44-7950741519

linkedin.com/in/dheerajsaxena

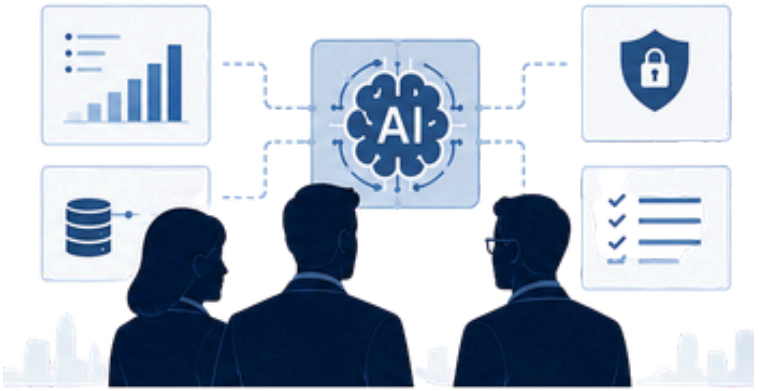
DHEERAJS@DATAWHISTL.COM

IBM

TCS TATA CONSULTANCY SERVICES

wipro

serco



WHAT THIS SESSION IS ABOUT

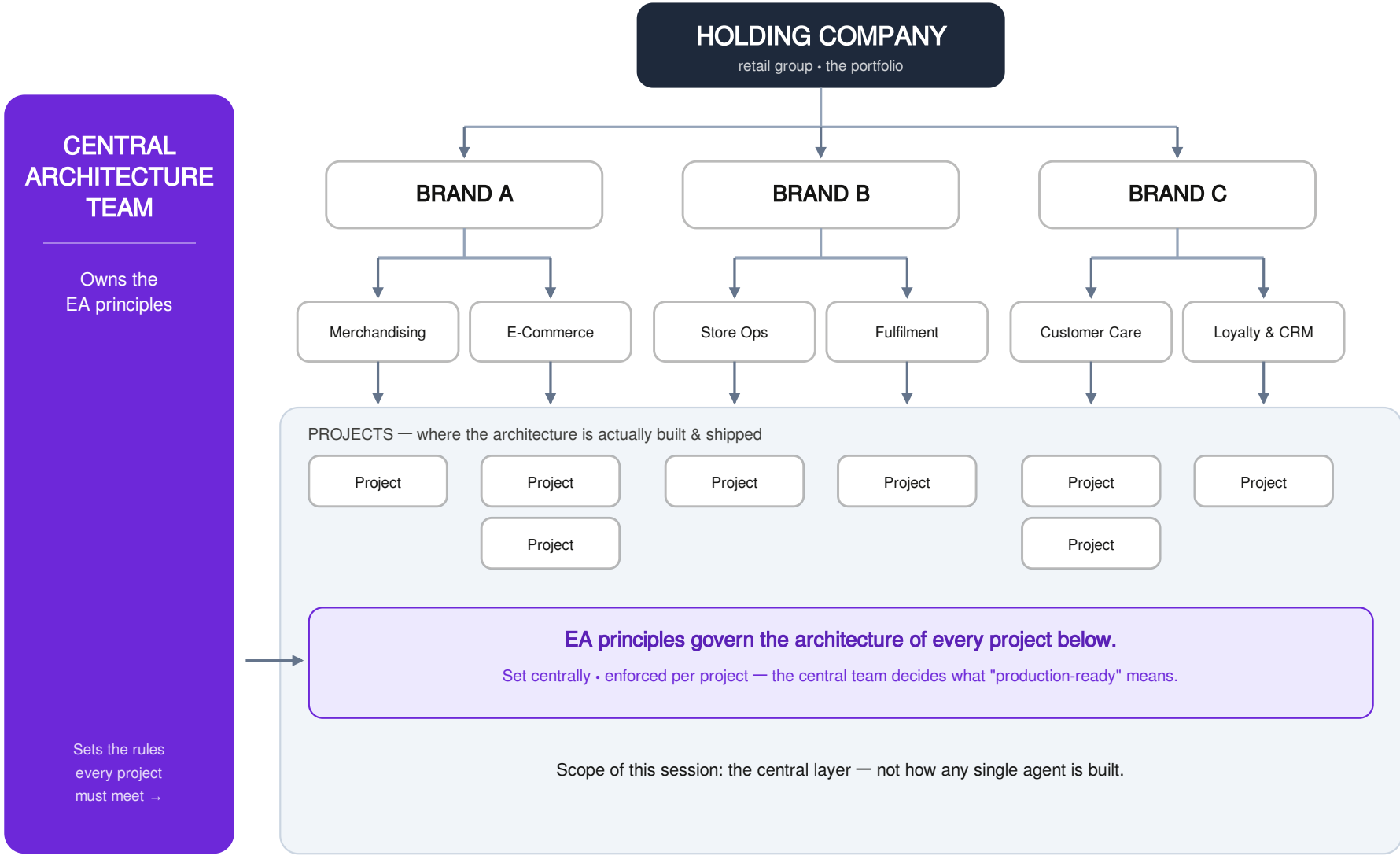
- Why enterprise AI fails after the demo: the cross-cutting non-functional concerns (cost, security, compliance, drift) that no single project owns or tests
- What turns it around: ***concrete, enforceable AI enterprise architecture standards*** — gated in CI, and strong governance

WHO THIS IS FOR

- Senior AI Developers, Architects, Project Managers
- Technical leaders moving AI past prototype into regulated production
- Anyone whose demo didn't survive contact with production

Intro Session (Today) → Hands-on workshops (July 2)

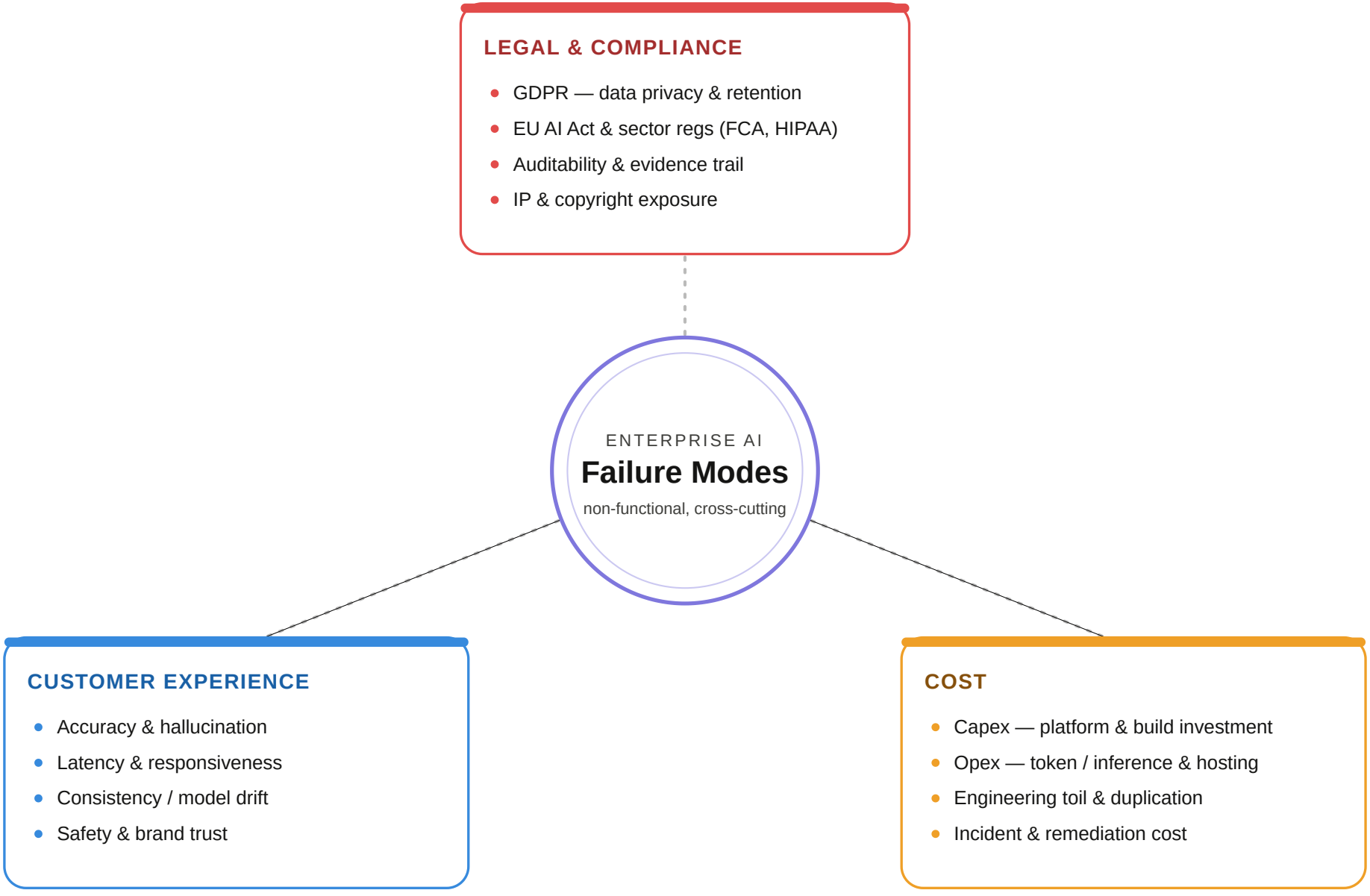
THE ENTERPRISE CONTEXT



THE ANATOMY OF A FAILURE

Enterprise AI — Non-Functional Failure Modes

The cross-cutting concerns projects don't test for themselves



EXAMPLE 1

Silent Regression — Uncommitted Eval Harness

Returns Triage Agent @ UK Retailer

The Setup

A major UK multi-channel retailer runs a Returns Triage Agent over ~2,000 returns/day across web and store. Each return is a customer-submitted form — order ID, item, reason and condition in the customer's own words. The agent reads the free-text reason ("don't fit lol the size was wierd, looks nothing like the picture"), pulls the order, checks payment status, prior returns and clearance, applies policy, and returns one of three decisions — **AUTO_APPROVE** (refund the card), **HUMAN_REVIEW** (escalate), or **REJECT** (clear violation). It replaced the call-centre human for straightforward cases and has been live for 6 months.

The Incident

A new release quietly starts issuing erroneous decisions — legitimate returns rejected, a surge of unexpected refunds. Nothing errors: CI is green and every test passed on the migration to production. The failure only surfaces through customer complaints and Finance flags, days after it shipped.

The Scale

The team investigates for two weeks, patches, ships. A week later an unrelated case fails the same way. They're permanently chasing yesterday's incident and never catch tomorrow's at PR time — every regression is found in production, by customers, not in review.

Why It Goes Unnoticed

CI Stays Green

The CI tests only cover happy paths, so they pass on every migration while production-realistic cases silently regress.

Knowledge walked out

Validation was done by a Senior AI Architect who left. His 50+ hand-validated production forms were never committed — the harness left with him.

No gate

Nothing replays production-realistic or adversarial cases at PR time, so a regression ships and stays shipped.

Root Cause & Fix

Commit a versioned, team-runnable eval harness (the golden set) to the repo and wire it as a required status check, so every PR replays production-realistic cases and a regression fails the build — not production. Quality lives in the repo, not the architect. (GENOPS01-BP01 → GO1B1-01)

EXAMPLE 2

Token Burn — Uncached Prompt Prefix

AI Support Chatbot @ P&L Insurer

The Setup

Every call to the model prepends the same large, stable block: a detailed system prompt, a fixed policy/product reference section, and a set of few-shot examples — several thousand tokens that never change between requests. Only the customer's question and the retrieved snippet vary. The provider bills full input-token price on the whole prompt every call, and offers a *cached-input* rate at a fraction of that price for a stable prefix — but nothing in the workload marks the prefix as cacheable, and caching was never turned on.

The Incident

There's no outage — the bot works perfectly. The failure is on the invoice. Because the multi-thousand-token static prefix is re-sent and re-billed at full rate on every single request, the workload pays full price, every call, for content that is identical every call.

The Scale

Multiplied across the chatbot's call volume, the avoidable spend is the prefix size times every request, day after day. The same answers, the same quality — at several times the token cost they'd pay with the stable prefix cached. Pure waste, growing linearly with traffic.

Why It Goes Unnoticed

Nothing breaks

Latency, error rate, answers — all fine. There's no signal except a line on the cloud bill.

The bill is diffuse

It shows up as "model spend," not "we re-billed the same 4k-token prefix a million times." No one's dashboard attributes it.

No gate

Caching is a config/prompt-structure decision nobody reviews at PR time, so an uncached reusable prefix ships and stays shipped.

Root Cause & Fix

Mark the cacheable static prefix of every reused prompt template and turn on prompt caching; fail the build when a cache-eligible template (prefix ≥ the model's minimum cache size) is left uncached or mis-declares its prefix size. The stable prefix is then billed at the cached rate; only the variable tail pays full price. (GENCOST03-BP03, GENCOST03-BP01)

EXAMPLE 3

Confident Fabrication — No Output Grounding Gate

Policy Q&A Agent @ UK Insurer

The Setup

A RAG support agent answers customer questions about cover, excess and renewals. For each question it retrieves snippets from the policy-document store and composes an answer. When retrieval returns nothing relevant — the question is outside the corpus, or only a thin match exists — the model doesn't stop. It produces a fluent, confident answer from its own parametric knowledge instead of saying "I don't know," and nothing checks that the answer is actually supported by the retrieved context. Live for 8 months.

The Incident

A customer asks about an edge case — accidental damage cover abroad. Retrieval returns weak, irrelevant snippets. The agent confidently states the policy covers it. It doesn't. The customer relies on the answer, travels, claims, gets rejected — and has a screenshot of the bot promising cover. | Every low-retrieval or out-of-corpus question is a coin-flip for a confident fabrication. It's invisible because the answers look authoritative; only the sliver that escalates to a complaint or the ombudsman ever surfaces. The rest erode trust silently, growing with traffic.

Why It Goes Unnoticed

Answers look authoritative

Fluent, on-brand, confident. A fabricated answer is visually identical to a correct one — there's no error and no "I'm not sure" to catch the eye.

Retrieval "worked"

The pipeline returned snippets and the model produced text. No exception, no empty result. The weak-match → fabrication path never raises a flag.

No gate

Nothing asserts the response is grounded in the retrieved context, or routes low-confidence retrieval to a safe fallback. So an ungrounded answer ships and stays shipped.

Root Cause & Fix

Route every response through a guardrail that checks grounding against the retrieved context (grounding-on for RAG), with a defined fallback when retrieval confidence is low or the answer isn't supported — and fail the build when a RAG workload's response path isn't wired through the grounding guardrail. The model may only assert what the context supports; everything else degrades to a safe "I can't confirm that — let me put you through." (GENSEC02-BP01 → GS2B1-01)

WHY THIS HAPPENS

Projects own functional requirements, not enterprise concerns

- A project is scoped, measured and rewarded on shipping its ***individual*** business outcome
- The cross-cutting NFRs — cost, security, compliance, drift — aren't explicitly in any project's remit.
- Person-dependent delivery culture- Quality lives in the architect, not the architecture.

No *enforceable* enterprise-wide AI Architecture Standards

- No shared definition of "production-ready." Project teams work in silos.
- Poor Architecture Governance. Nothing objective for the Architecture Review Board to enforce.

TURNING THE PATTERN AROUND

THE FIX

Make Production-Ready the Default

PRINCIPLES->STANDARDS->REFERENCE IMPLEMENTATIONS

WHAT YOU NEED

Two parts. Both required.

Part 1 — Enterprise-Wide AI Architecture Standards: *With Reference Implementations*

- A catalog of ***reference implementations*** (not just principles and standards)
- Objective basis for ARB, architects, and engineers to make and defend decisions.

Part 2 — Governance to Enforce Them

- Early involvement of AI Enterprise Architects.
- Pre-project architecture approvals.
- Milestone reviews.
- Pre-commit verifications before production deployment.

> Today's focus is **Part 1**: what principles are, what they prevent, and how to enforce them beyond relying solely on developer good-will.

How the reference implementation fixes it

Returns-triage regression • GENOPS01-BP01 → GO1B1-01 • governance certifies it, CI enforces it

THE STANDARD
Every production change must pass a versioned, team-runnable eval harness — committed to the repo — before it ships.

STEP 1 • BEFORE THE BUILD STARTS — approve, then build → STEP 2 • DURING THE BUILD • EVERY PR

ARB GOVERNANCE GATE

pre-build approval — the project self-certifies the audit sheet

Audit information sheet — the project declares & certifies:

- ☑ Golden set committed at the required path `eval/returns_golden.yaml`
- ☑ The eval runner implements the standard interface
- ☑ The eval check is wired as a REQUIRED status check
- ☑ Coverage includes production-realistic / adversarial cases

The project team signs it — and owns it.
If it ships broken later, the false certification is on them, not the ARB.

The ARB approves against the signed sheet — it does not crawl repos. Step 2 auto-detects whether the project actually did it.

CI/CD GATE

mechanical — auto-enforced on every PR, no human in the loop

- ✓ **Auto-detects the harness**
  Looks at the required location; missing or empty → build fails.
- ✓ **Enforces the interface**
  The runner must satisfy the standard contract, or it fails.
- ✓ **Replays on every PR**
  Runs the golden set; build FAILS the instant a case regresses.
- ✓ **Blocks the merge**
  Required status check via branch protection — can't merge red.

The certified controls hold mechanically.
The bad release fails the PR — not production.

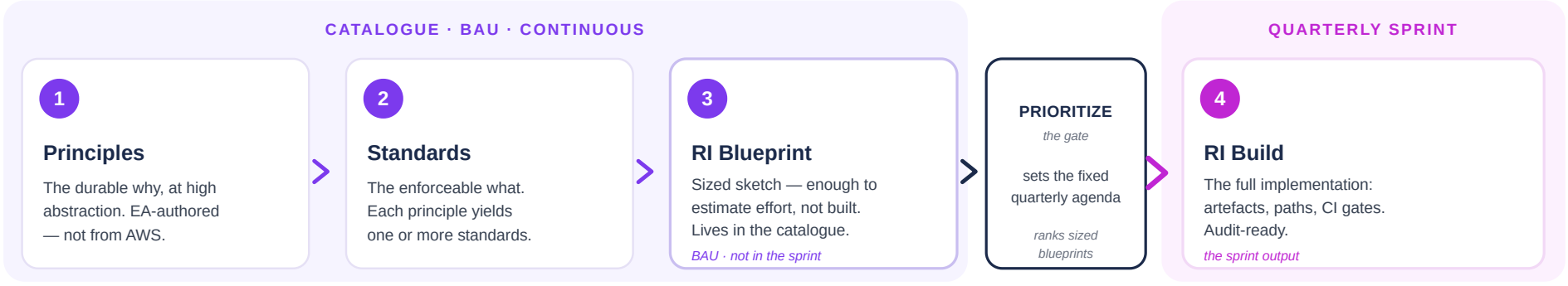
Governance certifies it up front • CI enforces it continuously • the project is accountable.

The ARB never inspects a repo — CI auto-detects compliance. Quality moves out of the person, into the architecture and the audit trail.

certify before build • enforce during build

How the EA Team Builds the Catalogue

Principles → Standards → RI Blueprint → Prioritize → RI Build. The catalogue runs continuously; builds run as quarterly sprints.



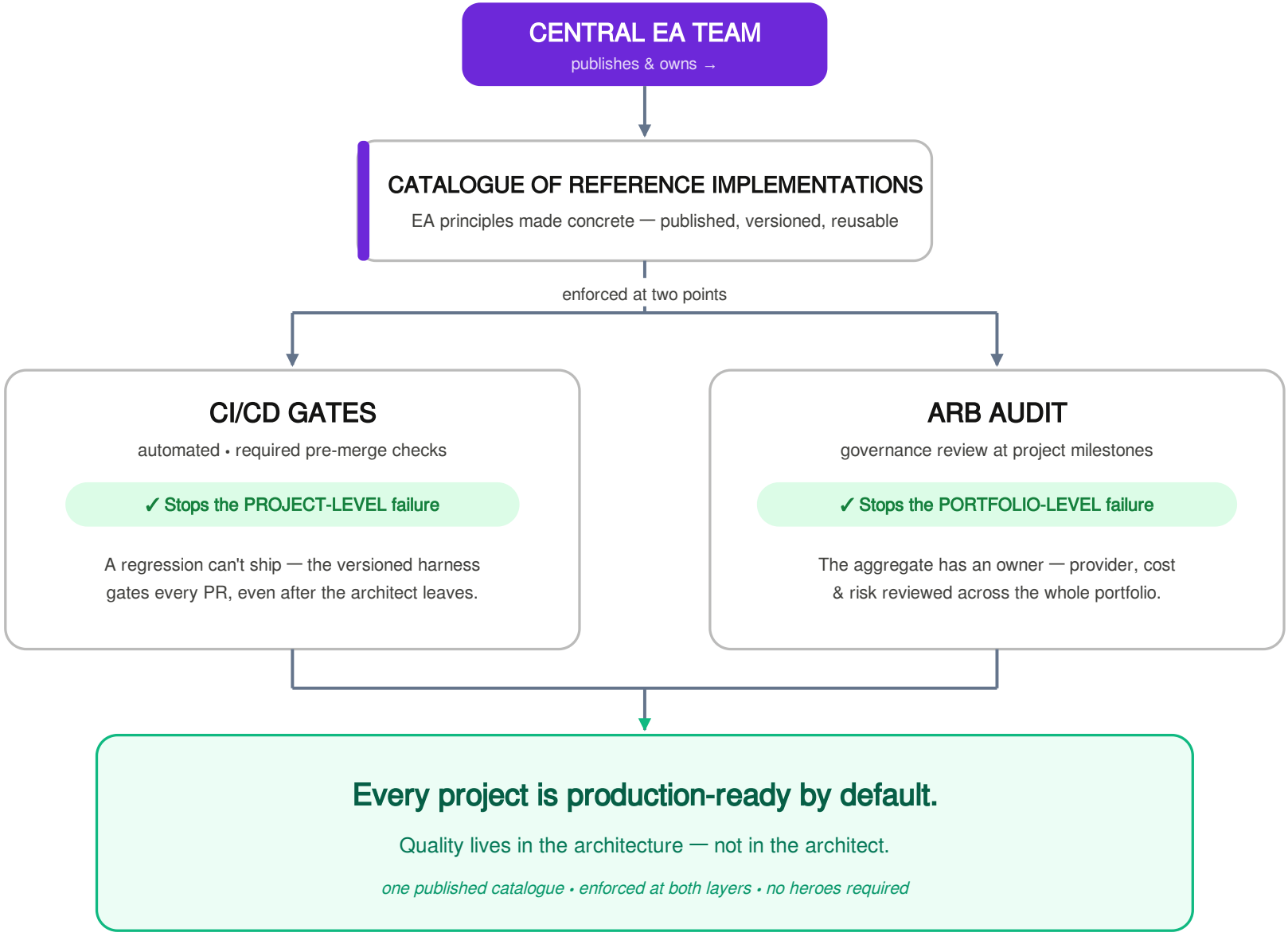
GOVERNANCE
Runs alongside

Review, approval, change control, RACI. Not architecture — but it's what keeps the catalogue alive. Different owners, a different cadence from the build team.

OPERATING MODEL — QUARTERLY SPRINTS

Each prioritised blueprint becomes one funded mini-project — its own budget, a named owner, a RACI chart — built out into a full reference implementation.

The catalogue carries principles, standards and sized blueprints. Prioritisation picks the quarter's builds — each ships as a funded, enforceable reference implementation.


CENTRAL EA TEAM
publishes & owns →
CATALOGUE OF REFERENCE IMPLEMENTATIONS
EA principles made concrete — published, versioned, reusable
enforced at two points
CI/CD GATES
automated • required pre-merge checks
✓ Stops the PROJECT-LEVEL failure
A regression can't ship — the versioned harness gates every PR, even after the architect leaves.
ARB AUDIT
governance review at project milestones
✓ Stops the PORTFOLIO-LEVEL failure
The aggregate has an owner — provider, cost & risk reviewed across the whole portfolio.
Every project is production-ready by default.
Quality lives in the architecture — not in the architect.
one published catalogue • enforced at both layers • no heroes required

This Session Was the *Why.*
The Workshop Is the *How.*

Today was the free intro. The build happens in the paid workshop.

Today · Intro Session

You saw the problem

- → Why we need AI architecture standards
- → How a principle becomes an enforceable implementation
- → What **needs** to be fixed

July 10

The Workshop
Hands-On
5 hrs

July 2 Workshop – What you get

Part 1 — The Problem

What counts as a production failure?

Before you can prevent failure, you have to agree on what it is. It falls into three categories.

Legal & Risk

Harm the business is liable for

- PII & sensitive-data leakage
- Regulatory non-compliance
- Unauthorized data access
- Harmful or prohibited output
- No audit trail or traceability

Customer Experience

Erosion of trust in the product

- Accuracy & quality drift
- Hallucinated or fabricated answers
- Inconsistent, off-brand tone
- Failed task completion
- Slow or unreliable responses

Cost

Spend that outruns the value

- Oversized-model token burn
- Runaway or unbounded usage
- Context & retrieval bloat
- Over-provisioned infrastructure
- No cost visibility or attribution

Three examples. Passing development isn't passing production —

and passing production still isn't success for the company. That third gap is why standards exist.

July 2 Workshop – What you get

Part 2 — Building the Catalogue

Where do the standards come from? Choosing what to baseline against.

1 Every standard serves one of three dimensions

Legal & Risk
what the business is liable for

Customer Experience
trust in the product

Cost
spend vs. value

2 No single source — you choose what to baseline against

Frameworks
AWS GenAI Lens,
NIST AI RMF,
ISO 42001

Regulation
EU AI Act, GDPR,
sector rules

Incidents
Post-mortems,
near-misses

Risk model
Threat modeling,
risk register

Bespoke
First-principles
(cost, CX)

No source covers all three dimensions — frameworks lean operational, regulation covers legal, cost & CX are usually bespoke. You compose.

3 A closer look at the framework we use: the AWS lens family

ML Lens
Full ML lifecycle — broadest scope

GenAI Lens
LLM · RAG · agentic — speaks our language

Responsible AI Lens
Fairness, safety, privacy — 8 focus areas

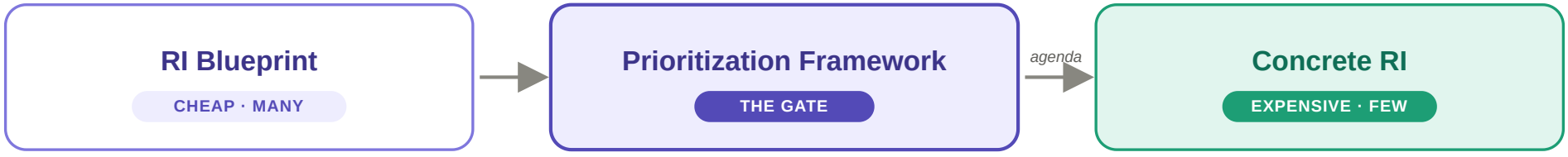
Well-Architected Framework — the foundation
Operational Excellence · Security · Reliability · Performance Efficiency · Cost Optimization · Sustainability

A lens re-applies the six pillars to one kind of workload. We baseline the catalogue on the GenAI Lens.

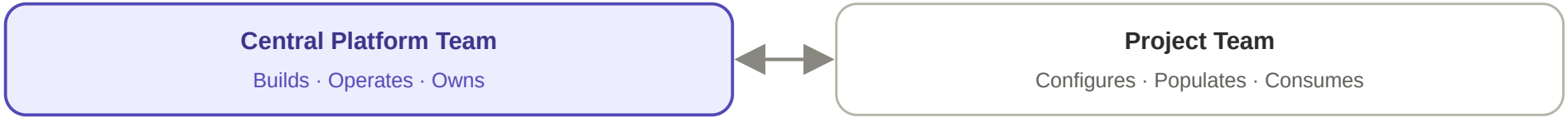
July 2 Workshop – What you get

Part 3 — The Process

Turning a candidate standard into something a team can actually build.



Then: who builds it, and who owns it?



A standard nobody owns is a standard nobody enforces.

We'll also touch on the operating-model org chart

Head of AI · Lead Architect · Head of Engineering · Head of Product · Head of Governance

How they all connect — that reveal is live in the session.

July 2 Workshop – What you get

Part 4 — Hands-On Lab

Walk the process for real — three standards, you build one.

1 Three standards — one per dimension

- **Legal & Risk** — *candidate standard*
- **Customer Experience** — *candidate standard*
- **Cost** — *candidate standard*

We walk all three through Blueprint → Prioritize → Concrete RI. You take one the whole way.

---

2 Build one from a sample taxonomy — the anatomy of a standard

We hand you a sample taxonomy and you fill in every field to develop the concrete build.

Statement	Problem	Solution	Maturity Level	Applicability	Criticality
Ownership	Enforcement Gates	Change Control	Framework Mappings	ARB Evidence	Reference Impl.

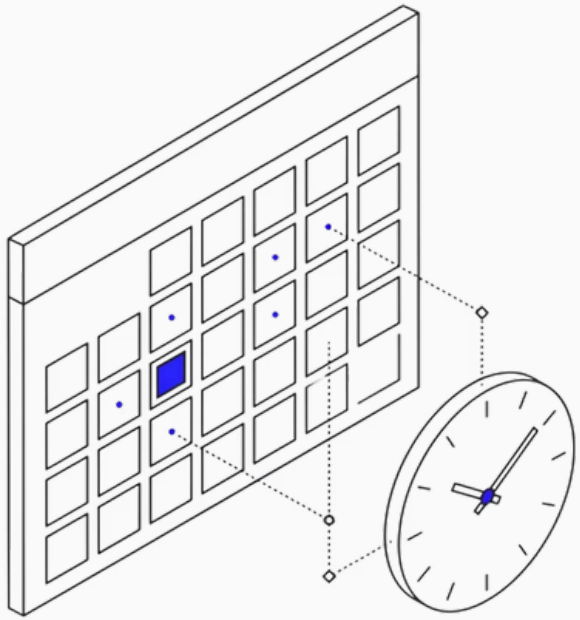
WHAT YOU WALK AWAY WITH

Concrete Reference Implementation

OWNED BY	BUILT BY	MAINTAINED BY
Central AI Factory	AI Engineering Platform Team	Central AI Product Team

✓ Automatically enforced at CI/CD gates

✓ Fully auditable



Choose Your Cohort

One day. Five hours. Production-ready architecture you can enforce from Monday morning.

July 2, 2026 3:00 – 8:00 PM GMT+1	**July 10, 2026** 12:00 – 5:00 PM GMT+1
July 17, 2026 4:00 – 9:00 PM GMT+1	**July 22, 2026** 2:00 – 7:00 PM GMT+1

$499 USD · Save 20%+ with Maven for Teams · Backed by the Maven Guarantee

Reserve Your Seat on Maven

Share on LinkedIn